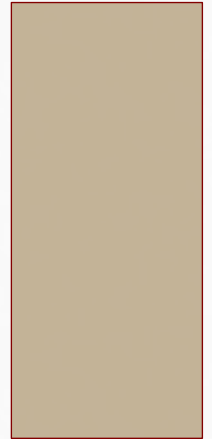


ROTATION MANIPULATION

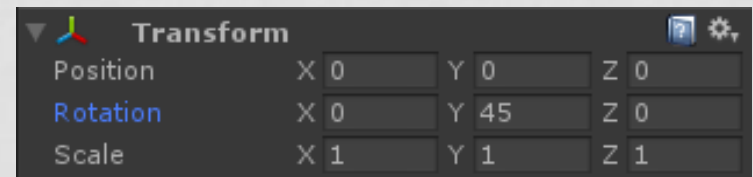
ADVANCED GAME DESIGN & DEVELOPMENT

DANNY JUGAN



ROTATIONS

- Angles come in two different types in Unity:
 - Quaternions
 - When you access `transform.rotation` of a game object, this is in the form of a quaternion
 - `Quaternion (x, y, z, w)`
 - Don't mess with these unless you know when you're doing.
 - Euler (pronounced Oiler) Angles
 - These are the angles that we're familiar with
 - `Euler (x, y, z)`
 - The values you see in the inspector window are in the form of Euler angles.
 - When you're manipulating rotations in Unity, you'll want to mess with Eulers, not Quaternions.



LOOKROTATION()

- Great for getting the rotation towards a point

Quaternion.LookRotation

```
static Quaternion LookRotation(Vector3 forward, Vector3 upwards = Vector3.up);
```

Parameters

forward	The direction to look in.
upwards	The vector that defines in which direction up is.

- Static function mean you use the class name to call them instead of an object of the class.
- `transform.rotation = Quaternion.LookRotation(target);`

LOOKROTATION()

- Exercise:
 - Given the two Game Objects declared below, write the line(s) of code that would make the boy face the girl.

```
public GameObject boy, girl;
```

```
Vector3 towards = girl.transform.position - boy.transform.position;  
boy.transform.rotation = Quaternion.LookRotation(towards);
```

- A common exception:
 - “Look rotation viewing vector is zero”

```
if(towards != Vector3.zero)  
    boy.transform.rotation = Quaternion.LookRotation(towards);
```

MANUALLY SETTING ROTATIONS

- There may be times in your game where you want to manually set the angle of an object.
- Unless you know how to use Quaternions, it will be difficult to use:
 - `transform.rotate = Quaternion;`
- Instead, you can use `transform.eulerAngles`.
 - `transform.eulerAngles = new Vector3(x, y, z);`

ROTATING VECTORS

- You can rotate a Vector by multiplying it by a Quaternion.
- Example:

```
//rotates the forward vector 45 degrees to the right  
Vector3 right45 = Quaternion.Euler(0, 45, 0) * transform.forward;
```

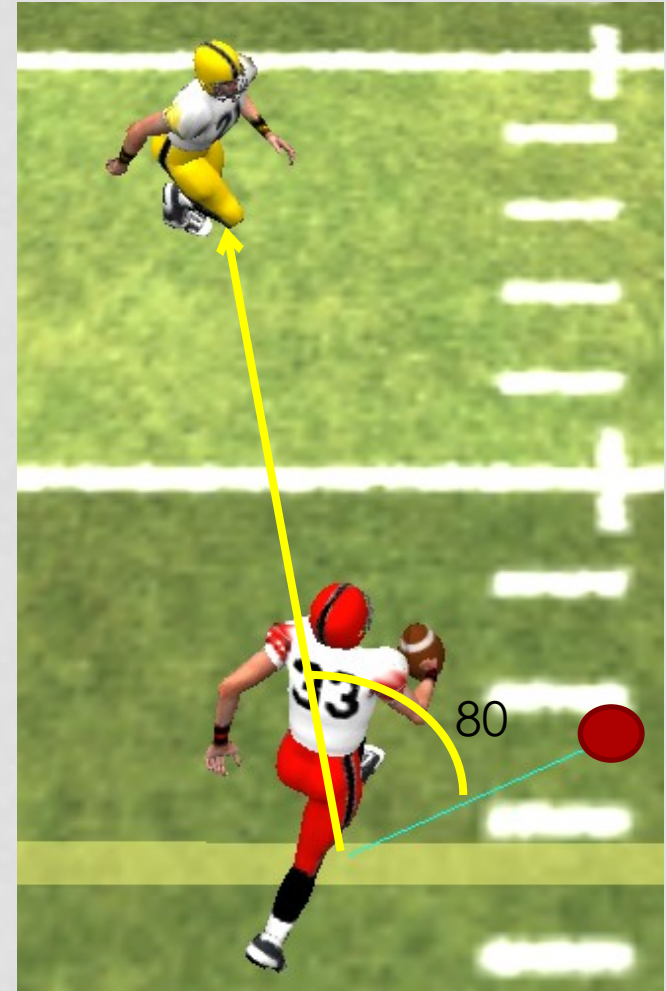
ROTATING VECTORS EXERCISE

Exercise:

Write the code to tell the ball holder (red player) to run 80 degrees to the right of the angle to the defender (yellow player).

```
public GameObject defender, ballHolder;
```

```
//target should end up at the red dot  
Vector3 target;
```



ROTATING VECTORS EXERCISE

```
public GameObject defender, ballHolder;  
Vector3 target;  
  
//get vector to defender  
Vector3 lookDir = defender.transform.position - ballHolder.transform.position;  
  
//normalize because we care only about the direction  
lookDir = lookDir.normalized;  
  
//rotate vector 80 degrees to the right  
target = Quaternion.Euler(0, 80, 0) * lookDir;  
  
//if you need to project the point out farther, multiply it by a value
```